Yelp Review Analysis: From Baselines to Business Intelligence Agent

A four-stage NLP pipeline progressing from TF-IDF baselines to fine-tuned transformers to agentic RAG systems

Github: github.com/YuhanWang93/Yelp-Project

Project Overview

This project applies a progressive four-stage NLP pipeline to the Yelp Business Dataset (50,000 reviews). Each stage builds on the previous, systematically quantifying computational cost against accuracy gains to evaluate whether more traditional feature engineering or a fully agentic retrieval augmented system.

Key progression: feature engineering → fixed pretrained representations → fine-tuned transformer fine-tuning → zero-/few-shot prompting → agentic retrieval-augmented systems

Stage 1 — Baseline Construction

Three modeling paradigms are compared on the 5-class star rating prediction task (test set).

Model	Accuracy	Macro F1
TF-IDF + Logistic Regression	56.92%	56.56%
Sentence-BERT + Logistic Regression	53.25%	52.89%
Dense Neural Network	**60.94%**	**62.58%**

Why does fine-tuned BERT win? End-to-end training jointly optimizes the text representation and the classification head for the exact task, allowing every layer to specialize. Frozen representations are fixed at inference time so any mismatch between the pre-training objective and our rating classification cannot be corrected.

Why does Sentence-BERT underperform TF-IDF? Sentence-BERT embeddings are optimized for semantic similarity, not sentiment discrimination. Two reviews with opposite sentiment but similar topic will cluster together in embedding space, causing a linear classifier to struggle. Lexical overlap (TF-IDF) is actually a strong signal for rating prediction.

Why does Dense Neural Network outperform Sentence-BERT? Even with frozen embeddings, the dense network learns a non-linear decision boundary over the embedding space, recovering some discriminative signal that a linear classifier cannot extract.

Stage 2 — BERT Fine-Tuning Experiments

A systematic hyperparameter search across **216 configurations** covers all combinations of the following dimensions:

Dimension	Values	Count
Backbone model	distilbert-base uncased , bert-base uncased , roberta-base	3
Task setting	Binary (2-class), 3-class, 5-class	3
Loss function	Cross-entropy, Weighted cross-entropy, Focal loss	3
Learning rate	1e-5, 2e-5	2
Batch size	4, 8, 16	3
Epochs	2, 3	2
Total combinations	**3 × 3 × 3 × 2 × 3 × 2**	**216**

Each run saves validation and test metrics, classification reports, confusion matrices, and prediction files. Due to compute cluster memory constraints, model checkpoints are not saved during Stage 2; only evaluation outputs are retained. Best 5-class configuration is retrained and saved in Stage 2 (Step 2).

Binary (2-class) — test set

Model	Best Accuracy	Best Macro F1	AUC
DistilBERT base uncased	96.54%	96.53%	—
BERT base uncased	97.07%	97.06%	—
RoBERTa base	**97.62%**	**97.61%**	**0.9968**

5-class — test set

Model	Best Accuracy	Best Macro F1	Off-by-1 Acc
DistilBERT base uncased	65.43%	65.36%	97.35%
BERT base uncased	66.75%	66.36%	97.49%
RoBERTa base	**68.56%**	**68.49%**	**98.10%**

Off-by-1 Accuracy explained: For ordinal classification, it counts predictions that are off by 1 (predict 4 if ground truth is 5, within ±1 star of the true label) e.g. predicting 3 when the true label is 5 is ✗ for Acc, but ✓ for Off-by-1. This metric captures near-miss predictions vs. catastrophic errors — a user who sees a restaurant ★★★ and instead predicts ★★★★ is still in the right neighborhood, expressing nearly the same sentiment. The consistently high Off-by-1 Accuracy (≥97% for all models) reflects the ordinal difficulty of distinguishing adjacent star ratings, where the linguistic signals in the review text are often genuinely ambiguous.

Why RoBERTa over BERT and DistilBERT? RoBERTa removes the Next Sentence Prediction task from pre-training entirely, dedicating all training signal to masked language modeling. This produces richer token representations that transfer more effectively to downstream tasks. DistilBERT, while faster, loses representation quality through knowledge distillation.

Why cb-focal beats mf-1 ? A smaller learning rate updates the pre-trained weights more gently, reducing catastrophic forgetting of linguistic knowledge. A small batch size introduces mild regularization that helps on imbalanced data. Focal loss down-weights easy examples and forces attention on hard, ambiguous reviews.

Why cb-focal underperforms on 3-class? Class-balanced focal loss down-weights majority classes (4 & 5-star reviews). In 3-class, these are merged into "positive", where sentiment signals are strongest. 3-class requires distinguishing adjacent ratings (e.g., 3 ★★★ vs. 4 ★★★★), which share similar vocabulary and are the most ambiguous. Down-weighting the majority class makes it directionally correct — the bottleneck is the ambiguity between neighboring star levels.

Best configuration (5-class): RoBERTa base, LR = 5 · batch = 4 · epochs = 3. This checkpoint is reused as the frozen encoder backbone for Stage 3.

Stage 3 — LLM Evaluation and Fine-Tuning

Two instruction-tuned LLMs are evaluated on the same rating task through a three-tier progression:

DeepSeek-7B (Instruct)	DeepSeek-R1-Distill-Qwen-14B
→ Tier 1: Zero-shot direct	→ Tier 1: Zero-shot direct
→ Tier 2: 1-shot few-shot (QLoRA)	→ Tier 2: 1-shot few-shot prompting
	→ Tier 3: LoRA fine-tune + thinking chains

Fine-Tuning Details

DeepSeek-7B (Instruct) — two hardware configurations:

Parameter	2Q-A100 (consumer GPU)	1(H)-(H)100 (80 GB)
Quantization	4-bit NF4 + double quant (bfloat16 compute)	None — full BF16
LoRA rank r	16	32
LoRA alpha	32	64
LoRA dropout	0.05	0.05
Target modules	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj
Learning rate	2e-4	2e-4
Epochs	2	2
Batch size (effective)	4 × 4 = 16	8 × 4 = 32
LR scheduler	Cosine, warmup ratio 3%	—
Max seq len (train)	256 tokens (Universe 1024)	—
Reported runtime	2.7 hrs	~1 hr (single throughput)

Both configurations apply LoRA adapters to all 7 projection layers (attention + FFN gate/up/down). Sequence packing is enabled during training to eliminate padding waste. The model is trained with cross-entropy loss; evaluation uses greedy decoding (temperature = 0) and strict output format parsing. Yelp review.

DeepSeek-R1-Distill-Qwen-14B — two stage fine-tuning pipeline:

Fine-tuning is restricted to reasoning traces preserving its chain-of-thought structure. A dedicated two-step pipeline is used:

Step	Input	Output	Purpose
1 — Generate thinking chains	Ground truth star + review text	<think>...</think>\nRating:	Structured CoT with reasoning chains
2 — LoRA fine-tune (SFT, A100)	Prompt + thinking chain + ground truth star	Rating:	LoRA adapter weights

LoRA hyperparameters for DeepSeek fine-tuning:

Parameter	2Q-A100	1(H)-(H)100
LoRA rank r	16	32
LoRA alpha	32	64
Target modules	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj
Learning rate	5e-5	4× lower than Qwen — preserves reasoning ability in distilled model
Epochs	2	1
Batch size (effective)	4 × 4 = 16	—
Max seq len (train)	1024	Thinking chains are much longer than plain reviews; 256 would truncate them

Training format for the assistant turn:

```
<think>
{teacher-generated reasoning chain}
</think>
Rating: {1 | 2 | 3 | 4 | 5}
```

Only samples where the base model already predicted the correct label are included — incorrect thinking chains are discarded to avoid reinforcing wrong reasoning.

DeepSeek Prompt Engineering — Tier 2 Improvement

DeepSeek-R1 requires an intermediate prompting step (Tier 2) that Qwen skips entirely. Zero-shot performance was severely limited by its insufficient token budget and the absence of anchor examples. The following changes raised 5-class accuracy from 55.68% to 61.56%:

Setting	Zero-shot	Improved Prompt
max_tokens	128	512
Few-shot example	None (zero-shot)	5 examples (one per star level)
Reasoning instruction	Unrestricted	"Reason briefly (2–3 sentences)"
Valid prediction rate	65.4%	~99%
5-class accuracy	55.68%	61.56%

Baseline prompt (max_tokens=512, 5-shot examples, brief reasoning):

```
You are a Yelp review rating classifier.
Analyze the review carefully.
Reason briefly (2-3 sentences) then output star (1-5).
Output format: Rating: <1|2|3|4|5>

Here are 5 examples:
"Terrible experience, never coming back." → Rating: 1
"Below average, mediocre at best." → Rating: 2
"Decent place, nothing special." → Rating: 3
"Really enjoyed it, will be back." → Rating: 4
"Absolutely fantastic! Best in the city!" → Rating: 5

Review: {text}
```

Why did each change improve accuracy?

- **max_tokens: 128 → 512:** Qwen-R1's chain-of-thought alone typically uses 100–300 tokens. With only 128 tokens, 35% of responses were truncated before the "Final rating:" line ever appeared, causing parse failures. Raising the budget to 512 directly recovered these valid predictions.
- **5 shot examples (one per star level):** Zero-shot prompts leave the model uncertain about how each rating feels in practice. Anchor examples calibrate the model's internal scale and reduce the scale-and-tone inconsistent output format, indirectly boosting parsing reliability.
- **"Reason briefly (2–3 sentences)":** Without a length constraint, the reasoning chain can still grow long enough to exhaust 512 tokens before the final answer. Capping reasoning to 2–3 sentences ensures the label always appears within budget.

Binary (2-class) — test set

Model	Method	Accuracy	Macro F1
DeepSeek-7B Instruct	Zero shot	96.10%	96.10%
DeepSeek-7B Instruct	**LoRA fine-tune (bf16)**	**98.55%**	**98.55%**
DeepSeek-R1-14B	Zero shot	89.04%	79.41%
DeepSeek-R1-14B	Few shot (improved)	91.24%	87.23%
DeepSeek-R1-14B	LoRA fine-tune (bf16)	98.42%	98.42%

5-class — test set

Model	Method	Accuracy	Macro F1	Off-by-1
DeepSeek-7B Instruct	**LoRA fine-tune (bf16)**	**68.58%**	**68.27%**	**98.04%**
DeepSeek-7B Instruct	Zero shot	67.36%	66.91%	98.15%
DeepSeek-R1-14B	LoRA fine-tune (bf16)	67.70%	66.67%	98.44%
DeepSeek-R1-14B	Few shot (improved)	61.56%	60.90%	98.04%
DeepSeek-R1-14B	Zero shot	55.68%	54.48%	89.93%

Why is Qwen zero-shot already near fine-tuned best? Qwen2.5-7B Instruct was pre-trained on a large corpus containing review-like text and instruction following data. It already understands the relationship between review language and star ratings, so fine-tuning primarily sharpens output formatting rather than teaching new reasoning skills. Its strong baseline suggests the model's capability ceiling for this task, not a limitation of the tuning.

Why did DeepSeek-R1 at zero-shot (DeepSeek-R1 is a reasoning model that generates an extended chain-of-thought before its final answer) **lag so much?** With an default `max_tokens` budget, the reasoning chain ran out of mid-way, and the model never reached a final label. This is a general inference failure, not a capability failure — after increasing the token budget and adding few-shot examples, 5-class accuracy jumped from 55.68% to 61.56%.

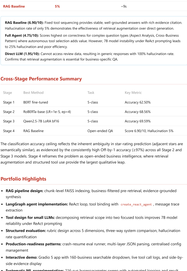
Why does LoRA fine-tuning work effectively? LoRA freezes the base model weights and learns only small low-rank update matrices, preserving pre-trained linguistic knowledge while adapting to the task distribution. For strong base models, LoRA primarily achieves format alignment (e.g. outputting a single digit instead of a verbose rather than a structured classification) and calibration, rather than teaching new reasoning capabilities.

Why does fine-tuning improve DeepSeek-7B but hurt DeepSeek-R1-14B? Fine-tuning DeepSeek-7B Instruct (+68% relative) works because its chain-of-thought capability is latent — the model knows how to reason but defaults to direct answers. Fine-tuning teaches it when to activate that reasoning. For DeepSeek-R1-14B, the reasoning capability is already trained in and actively useful. Fine-tuning on ground-truth labels can interfere with this pre-trained reasoning ability, degrading its reasoning ability. The gap between fine-tuned DeepSeek-7B and fine-tuned R1-14B (68.58% vs. 67.70%) is within noise, so the 7B model is more cost-efficient for this task.

Stage 4 — Business Intelligence Agent

Stage 4 shifts from classification to open-ended question answering over Yelp review data. A two-layer system is built and evaluated against a no-retrieval LLM baseline.

System Architecture



Technical Stack

Langgraph state machine, agent loop	FAISS IndexFlat2 , all-MiniLM-L6-v2 , RoBERTa base classifier
Langchain + langchain-openai , tenacity	vLLM Server (Qwen2.5-7B-Instruct)

Vector Knowledge Base

- 50,000 Yelp reviews — **82,605 vectors** (sentence-split, 1,500 chunks max)
- Encoded with `all-MiniLM-L6-v2` (384-dim, L2-normalized — cosine similarity via inner product)
- FAISS `IndexFlatIP` for exact nearest-neighbor search
- Business-filtered retrieval: subset matrix multiply on `business_to_indices` map (avoids result dilution for small businesses)

Evaluation Setup

20 questions × 4 tapes × 3 systems = 60 evaluated responses. Human scored on 5 dimensions (0–2 scale, max 10 points):

Question Type	Example Question
Complaint Mining	"What do customers complain about most at this business?"
Aspect Analysis	"What aspects do customers praise and criticize about food and service?"
Business Profiling	"Give me an overall profile of this business based on customer reviews."
Cross-Business Pattern	"Do Yelp reviews show any patterns between sale time and star ratings?"

Evaluation Results

System	Factuality	Evidence Quality	Grounedness	Bias Awareness	Total (10)
Direct LLM	0.25	0.00	0.00	1.75	1.95
RAG Baseline	**0.95**	**1.60**	**1.75**	0.95	**5.25**
Full Agent	1.05	1.15	1.15	**1.80**	5.15

System	Hallucination Rate	Avg Response Time
Direct LLM	0%	~8s
Full Agent	25%	~45s
RAG Baseline	**5%**	**~8s**

RAG Baseline (8.90/10): Fixed tool sequencing provides stable, well-grounded answers with evidence citation. Hallucination rate of 5% demonstrates the effectiveness of retrieval augmentation over direct generation.

Full Agent (74.75%): Scores higher on correctness for complex question types (Aspect Analysis, Cross-Business Pattern) where autonomous tool selection adds value. However, 79 model invocations per run (vs. 3 for baseline) leads to 25% hallucination and poor efficiency.

Direct LLM (19.75%): Cannot cite evidence or ground claims in specific reviews with 100% hallucination rate. Confirms that retrieval augmentation is essential for business specific QA.

Cross-Stage Performance Summary

Stage	Best Method	Task	Key Metric
Stage 1	BERT fine-tuned	5-class	Accuracy 60.94%
Stage 2	RoBERTa base (LR=5e-5, epochs=3)	5-class	Accuracy 68.56%
Stage 3	Qwen2.5-7B LoRA SFT	5-class	Accuracy 68.58%
Stage 4	RAG Baseline	Open ended QA	Score 8.90/10, Hallucination 5%

The classification scaling laws reflect the inherent ambiguity in star rating prediction (adjacent stars are semantically similar), as evidenced by the consistently high Off-by-1 accuracy (≥97%) across all Stage 2 and Stage 3 models. Stage 4 reframes the problem as open-ended business intelligence, where retrieval augmentation and structured tool use provide the largest qualitative leap.

Portfolio Highlights

- **RAG pipeline design:** chunk-level FAISS indexing business filtered pre-retrieval, evidence grounded synthesis
- **Langgraph agent implementation:** ReAct loop, tool binding with `create_react_agent` , message trace extraction
- **Tool design for small LLMs:** decomposing retrieval scope into two focused tools improves 7B model reliability under ReAct prompting
- **Structured evaluation:** rubric design across 5 dimensions, three system comparison, hallucination rate quantification
- **Production readiness patterns:** crash-resume eval runner, multi-layer JSON parsing, centralized config management
- **Interactive demo:** Gradio 5 app with 100 business selectable dropdown, live tool call logs, and side-by-side evidence display
- **Systematic ML experimentation:** 216 hyperparameter sweep with automated logging and result aggregation (Stage 2)